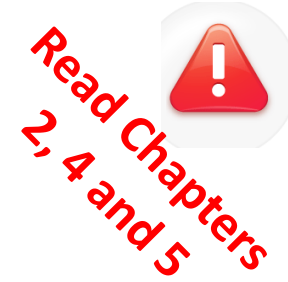


CMPS 251



Unit 1_02

Java Fundamentals

Writing Simple Programs

Outline

- Java Program
- Variables
- Data Types
- Arithmetic Operations
- Constant
- String Class
- Input / Output
- Packages

First Java Program



First Program in Java: Printing a Line of Text

```
1  // Fig. 2.1: Welcome1.java
2  // Text-printing program.
3
4  public class Welcome1 {
5     // main method begins execution of Java application
6     public static void main(String[] args) {
7        System.out.println("Welcome to Java Programming!");
8     } // end method main
9  } // end class Welcome1
```

```
Welcome to Java Programming!
```

Fig. 2.1 | Text-printing program.

First Program in Java: Printing a Line of Text

Commenting Your Programs

- `// Fig. 2.1: Welcome1.java`
- `// indicates that the line is a comment.`
- Used to document programs and improve their readability.
- Compiler ignores comments.
- A comment that begins with **//** is an end-of-line comment
- Traditional comment, can be spread over several lines as in
- ```
  /* This is a traditional comment. It
     can be split over multiple lines */
  ```
- This type of comment begins with `/*` and ends with `*/`.
- All text between the delimiters is ignored by the compiler.
  ```

First Program in Java: Printing a Line of Text

Using Blank Lines

- Blank lines, space characters and tabs
  - Make programs easier to read.
  - Together, they’re known as white space (or whitespace).
  - **White space is ignored by the compiler.**
  - Used to document programs and improve their readability.

First Program in Java: Printing a Line of Text

Declaring a class

- Class declaration

```
public class Welcome1
```

  - Every Java program consists of at least one class that you define.
  - class keyword introduces a class declaration and is immediately followed by the class name.
  - **Keywords** are reserved for use by Java and are always spelled with all lowercase letters.

First Program in Java: Printing a Line of Text

Java Reserved Keywords

abstract	double	instanceof	static
assert	else	int	strictfp
boolean	enum	interface	super
break	extends	long	switch
byte	false	native	synchronized
case	for	new	this
catch	final	null	throw
char	finally	package	throws
class	float	private	transient
const	goto	protected	true
continue	if	public	try
default	implements	return	void
do	import	short	volatile
			while

First Program in Java: Printing a Line of Text

Filename for a public Class

- A public class must be placed in a file that has a name of the form **ClassName.java**, so class **Welcome1** is stored in the file **Welcome1.java**.

Class Body

- A left brace, **{**, begins the body of every class declaration.
- A corresponding right brace, **}**, must end each class declaration.

First Program in Java: Printing a Line of Text

`System.out` object

- Standard output object.
  - Allows a Java application to display information in the **command window** from which it executes.

`System.out.println` method

- Displays (or prints) a line of text in the command window.
  - The string in the parentheses the argument to the method.
  - Positions the output cursor at the beginning of the next line in the command window.
- Most statements end with a semicolon.

First Program in Java: Printing a Line of Text

Compiling Your First Java Application

- To compile the program, type

```
javac Welcome1.java
```

  - If the program contains no compilation errors, preceding command creates a .class file (known as the class file) containing the platform-independent Java bytecodes that represent the application.
  - When we use the java command to execute the application on a given platform, these bytecodes will be translated by the JVM into instructions that are understood by the underlying operating system.

First Program in Java: Printing a Line of Text

Executing the Welcome1 Application

- To execute this program in a command window, change to the directory containing Welcome1.java
- C:\examples\ch02\fig02_01 on Microsoft Windows or
- ~/Documents/examples/ch02/fig02_01 on Linux/macOS.
- Next, type **java Welcome1**
- This launches the JVM, which loads the **Welcome1.class** file.
- The command omits the .class file-name extension; otherwise, the JVM will not execute the program.
- The JVM calls class Welcome1's main method.

First Program in Java: Printing a Line of Text

Executing the Welcome1 Application

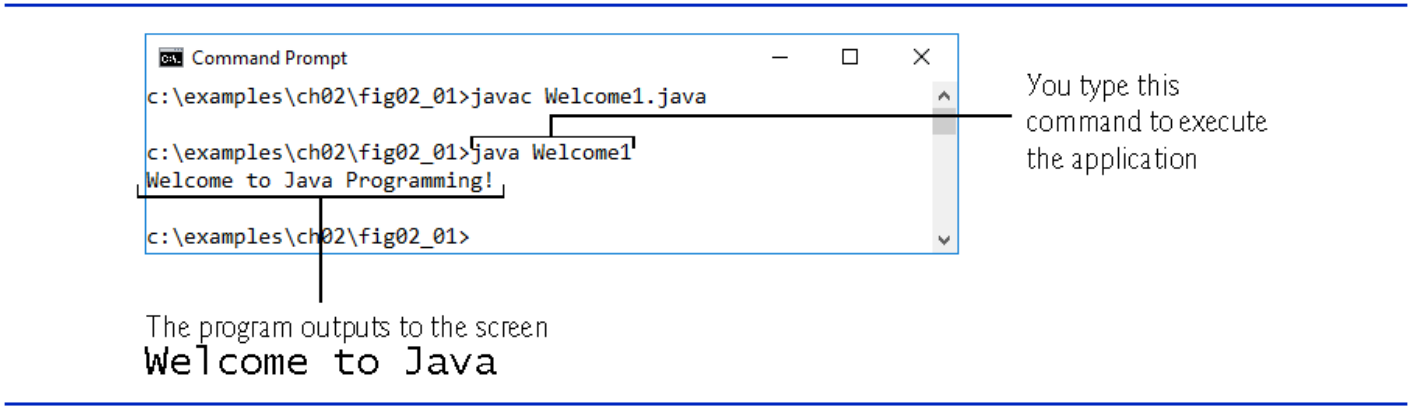


Fig. 2.2 | Executing `Welcome1` from the **Command Prompt**.

Modifying Your First Java Program

- System.out's method print displays a string.
- Unlike println, print does not position the output cursor at the beginning of the next line in the command window.

```
1  // Fig. 2.3: Welcome2.java
2  // Printing a line of text with multiple statements.
3
4  public class Welcome2 {
5     // main method begins execution of Java application
6     public static void main(String[] args) {
7        System.out.print("Welcome to ");
8        System.out.println("Java Programming!");
9     } // end method main
10 } // end class Welcome2
```

```
Welcome to Java Programming!
```

Fig. 2.3 | Printing a line of text with multiple statements.

Modifying Your First Java Program

- Newline characters indicate to System.out's print and println methods when to position the output cursor at the beginning of the next line in the command window.
- Newline characters are whitespace characters.
  - The backslash (\) is called an **escape character**.
  - Indicates a "*special character*"
  - Backslash is combined with the next character to form an **escape sequence**

    \n represents the newline character.

Modifying Your First Java Program

```
1  // Fig. 2.4: Welcome3.java
2  // Printing multiple lines of text with a single statement.
3
4  public class Welcome3 {
5     // main method begins execution of Java application
6     public static void main(String[] args) {
7        System.out.println("Welcome\nto\nJava\nProgramming!");
8     } // end method main
9  } // end class Welcome3
```

```
Welcome
to
Java
Programming!
```

Fig. 2.4 | Printing multiple lines of text with a single statement.

Escape Sequences

Escape sequence	Description
`\n`	Newline. Position the screen cursor at the beginning of the *next* line.
`\t`	Horizontal tab. Move the screen cursor to the next tab stop.
`\r`	Carriage return. Position the screen cursor at the beginning of the *current* line—do *not* advance to the next line. Any characters output after the carriage return *overwrite* the characters previously output on that line.
`\\`	Backslash. Used to print a backslash character.
`\"`	Double quote. Used to print a double-quote character. For example, `System.out.println("\"in quotes\"");` displays `"in quotes"`.

Fig. 2.5 | Some common escape sequences.

Variables and Literals

- A variable is a named storage location in the computer's memory.
- A literal is a value that is written into the code of a program.
- Programmers determine the number and type of variables a program will need.

```
public class Variable {
public static void main(String[] args) {
        int value; //declaration of a variable
        value = 5; // value assignment to the variable
        System.out.print("The value is ");
        System.out.println(value);
        }
}
//The value is 5
```

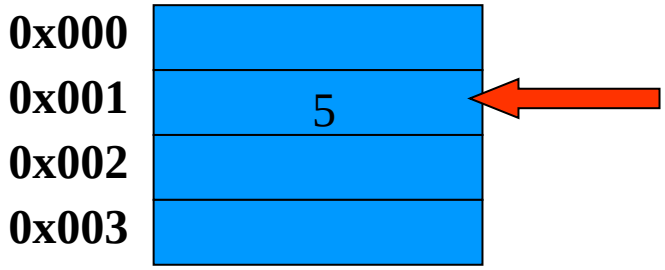
Variables and Literals

This line is called a *variable declaration*.

```
int value;
```

The following line is known as an assignment statement.

```
value = 5;
```



The value 5 is stored in memory.

This is a string *literal*. It will be printed as is.

```
System.out.print("The value is ");
System.out.println(value);
```

The integer 5 will be printed out here.
Notice no quote marks?

The + Operator

- The + operator can be used in two ways.
  - as a **concatenation** operator
  - as an **addition** operator
- If either side of the + operator is a string, the result will be a **string**.

```
System.out.println("Hello " + "World");
System.out.println("The value is: " + 5);
System.out.println("The value is: " + value);
System.out.println("The value is: " + '\n' + 5);
```

String Concatenation

- Java commands that have string literals must be treated with care.
- A string literal value cannot span lines in a Java source code file.

```
System.out.println("This line is too long and now it
has spanned more than one line, which will cause a
syntax error to be generated by the compiler. ");
```

- Fix with "+"

```
System.out.println("These lines are " +
                   "are now ok and will not " +
                   "cause the error as before.");
```

- Can join various data types.

```
System.out.println("We can join a string to " +
                   "a number like this: " + 5);
```

String Concatenation

- Can be used to format complex String objects.

```
System.out.println("The following will be printed " +
                   "in a tabbed format: " +
                   "\n\tFirst = " + 5 * 6 + ", " +
                   "\n\tSecond = " + (5 * 6) + ", " +
                   "\n\tThird = " + 16.7 + ".");
```

if an addition operation is also needed, it must be put in parenthesis

Output →

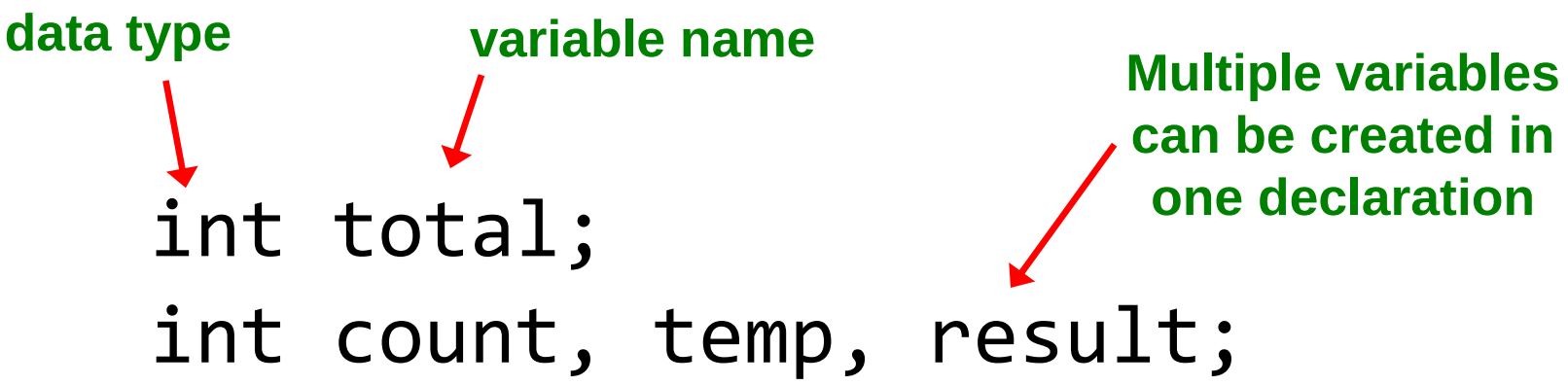
```
The following will be printed in a tabbed format:
        First = 30,
        Second = 30,
        Third = 16.7.
```

Identifiers

- Identifiers are programmer-defined names for:
  - Classes , variables , methods
- Identifiers may not be any of the Java reserved keywords.
- Identifiers must follow certain rules, it may only contain:
  - letters a-z or A-Z,
  - the digits 0-9,
  - underscores (_), or
  - the dollar sign ($)
- **The first character may not be a digit.**
- Identifiers are case sensitive.
  - itemsOrdered is not the same as itemsordered.
- Identifiers cannot include spaces.

Variables

- A variable must be *declared* by specifying its **name** and the **type** of data that it will hold





Always choose meaningful and descriptive variable names

Variable Names

- Variable names should be descriptive.
- Descriptive names allow the code to be more readable; therefore, the code is more maintainable.
- Which of the following is more descriptive?

```
double tr = 0.0725;
```

```
double salesTaxRate = 0.0725;
```

- Java programs should be self-documenting.

Java Naming Conventions

- **Variable** names should **begin with a lower case letter** and then switch to title case thereafter:
  - Ex: int **caTaxRate**
- **Class** names should be all **title case.**
  - Ex: public class **BigLittle**
- More Java naming conventions can be found at:
  - http://java.sun.com/docs/codeconv/html/CodeConventions.doc8.html

Primitive Data Types



Primitive Data Types

- Primitive data types are built into the Java language and are not derived from classes. There are 8 Java primitive data types.

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Variable Declarations

- Variable Declarations take the following form:

DataType *VariableName*;

```
byte inches;
short month;
int speed;
long timeStamp;
float salesCommission;
double distance;
```

Integer Data Types

- **byte**, **short**, **int**, and **long** are all integer data types.
- They can hold whole numbers such as 5, 10, 23, 89, etc.
- Integer data types cannot hold numbers that have a decimal point in them.

Floating Point Data Types

- Data types that allow fractional values are called **floating-point** numbers.
  - 1.7 and -45.316 are floating-point numbers.
- In Java there are two data types that can represent floating-point numbers.
  - **float** - also called single precision (7 decimal points).
  - **double** - also called double precision (15 decimal points).
- They can hold whole numbers such as 5, 10, 23, 89, etc.
- Integer data types cannot hold numbers that have a decimal point in them.

Floating Point Literals

- When floating point numbers are embedded into Java source code they are called floating point literals.
- The default type for floating point literals is **double**.
  - 29.75, 1.76, and 31.51 are **double** data types.
- **A double value is not compatible with a float variable because of its size and precision.**

```
float number;
number = 23.5; // Error!
```

- A double can be forced into a float by appending the letter **F** or **f** to the literal.

```
float number;
number = 23.5F; // This will work.
```

Scientific and E Notation

- Floating-point literals can be represented in scientific notation.

Decimal Notation	Scientific Notation	E Notation
247.91	2.4791×10^{2}	2.4791E2
0.00072	7.2×10^{-4}	7.2E-4
2,900,000	2.9×10^{6}	2.9E6

The boolean Data

- The Java **boolean** data type can have two possible values.
  - true
  - false
- The value of a boolean variable may only be copied into a boolean variable.

```
boolean done = false;
```

The char Data Type

- The Java char data type provides access to single characters.
- Don’t confuse char literals with string literals.
  - **char** literals are enclosed in **single quotes**.
  - **String** literals are enclosed in **double quotes**.
- Character literals are delimited by single quotes:

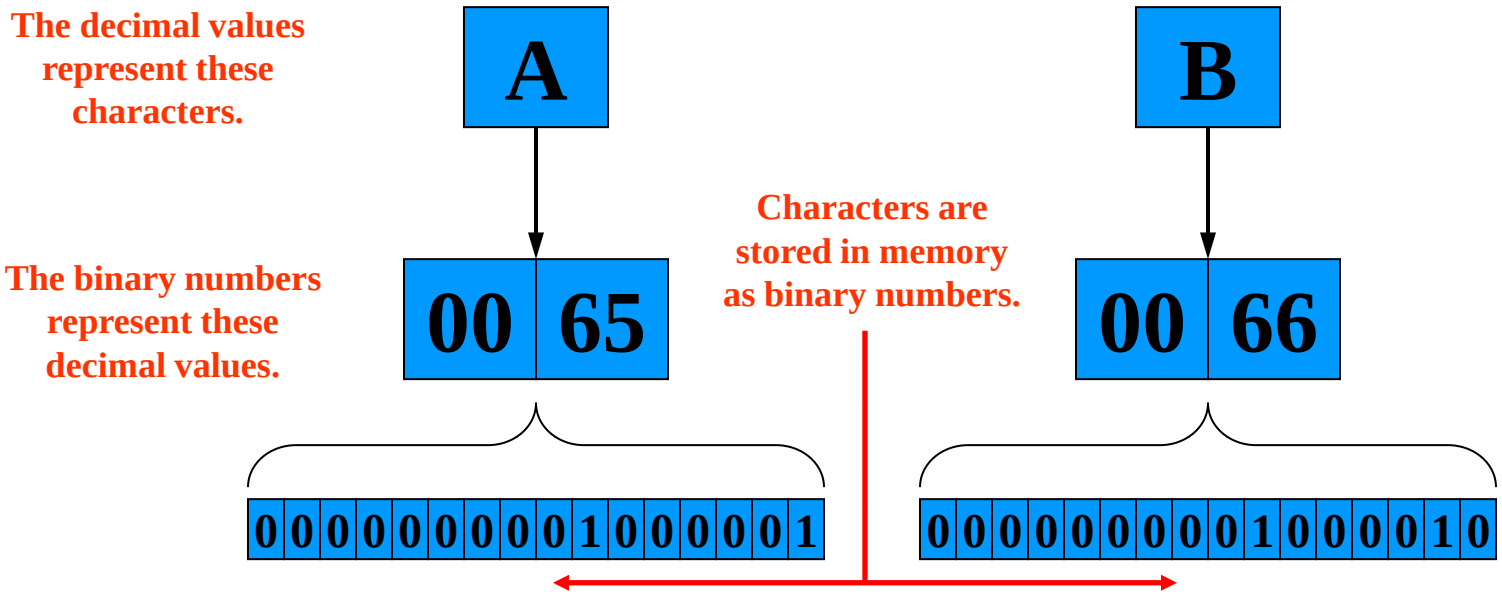
  'a' 'X' '7' '$' ',' '\n'
- Example declarations

```
char topGrade = 'A';
char terminator = ';', separator = ' ';
```

Unicode

- Internally, characters are stored as numbers.
- **Each character takes up 2 bytes in memory**.
- The first 256 characters in the Unicode character set are compatible with the ASCII* character set.

*American Standard Code for Information Interchange



Variable Assignment and Initialization

- In order to store a value in a variable, an assignment statement must be used. The assignment operator is the equal (=) sign.
  - Left side of the assignment operator must be a variable name.
  - **Right side must be either a literal or expression that evaluates to a type that is compatible with the type of the variable**.

```
int month; // Once declared, they can then receive a value
month = 2; //After receiving a value, variables can then be used
int days = 28; //Can be declared and initialized on he same line.
System.out.println("Month " + month + " has " + days + " Days.");
```

Implicit type using var

NEW

- Java 10 and above has **var** keyword to declare a variable without explicit type

  e.g. instead of doing

```
String str = "Java" ;
```

  You can just say

```
var str = "Java" ;
```

- Java will **implicitly** recognize the variable data type based on the initial value assigned to it
- This will be heavily used in this course!

animation

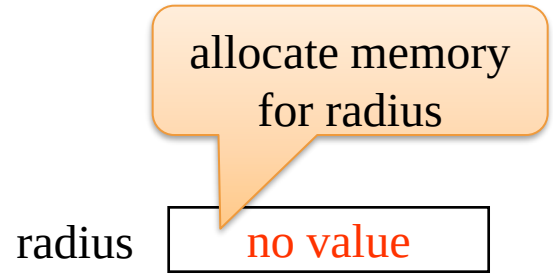
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the
      circle of radius " +
      radius + " is " + area);
  }
}
```



animation

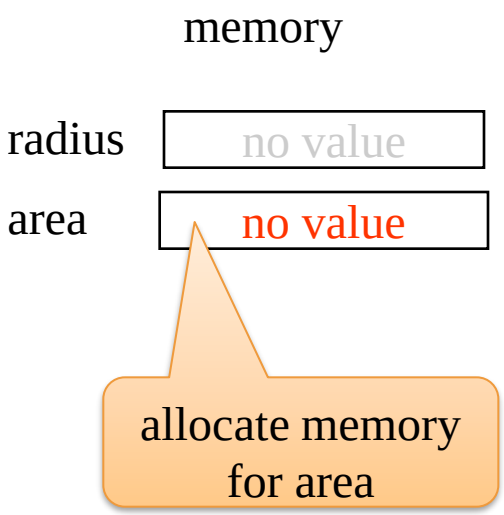
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the
      circle of radius " +
      radius + " is " + area);
  }
}
```



animation

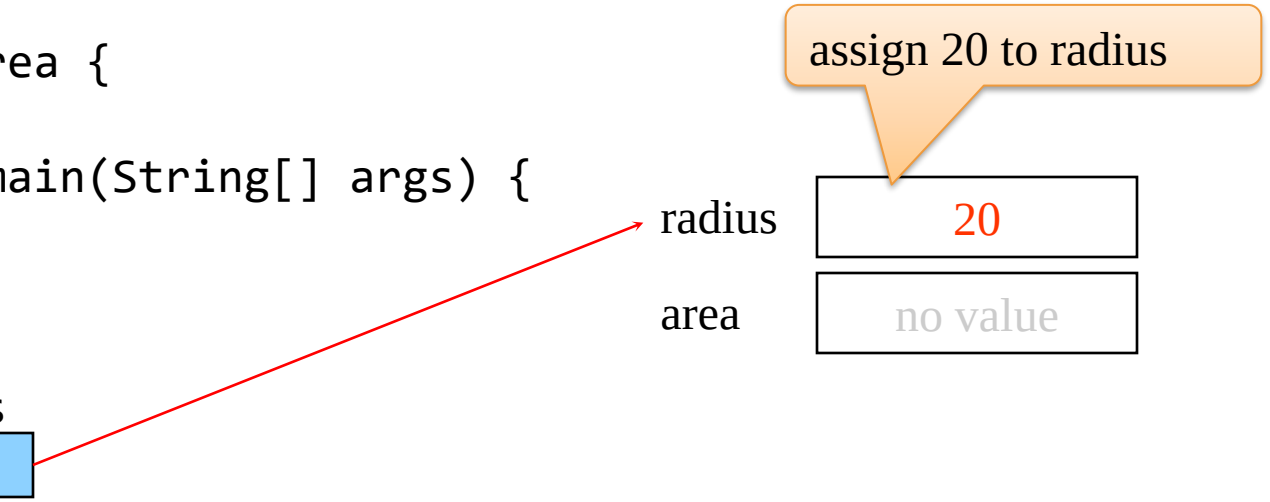
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the
   circle of radius " +
      radius + " is " + area);
  }
}
```



animation

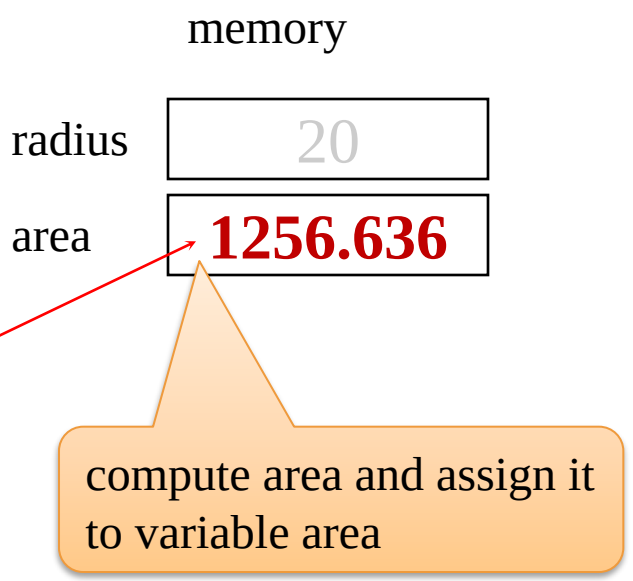
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



animation

Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```

memory

radius	20
area	1256.636

print a message to the console



Arithmetic Operations



Arithmetic Operators

Operator	Meaning	Type	Example
+	Addition	Binary	`total = cost + tax;`
-	Subtraction	Binary	`cost = total - tax;`
*	Multiplication	Binary	`tax = cost * rate;`
/	Division	Binary	`salePrice = original / 2;`
%	Modulus	Binary	`remainder = value % 5;`

Arithmetic Operators

- Each operator must have a left and right operand.
- The arithmetic operators work as one would expect.
  - It is an error to try to divide any number by zero.
  - When working with two integer operands, the division operator requires special attention
- **Integer Division**
  - In a Java program, what is the value of 1/2?
    - You might think the answer is 0.5…
    - But, that's wrong.
    - The answer is simply 0.
    - Integer division will truncate any decimal remainder.

Operator Precedence

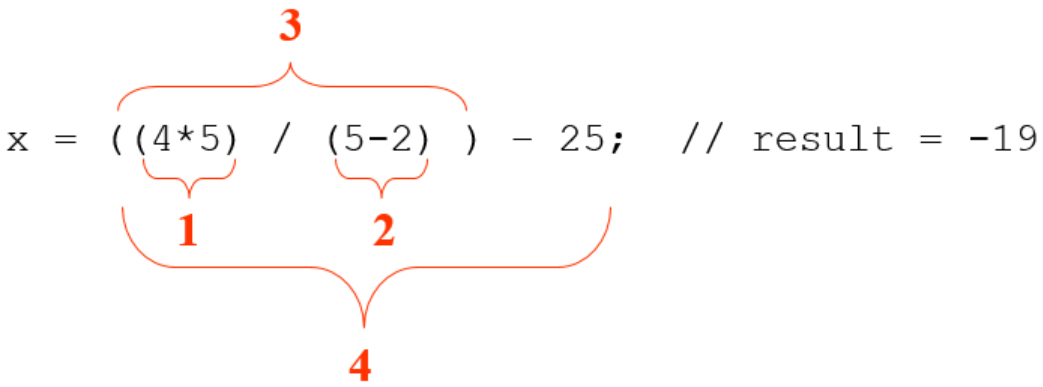
Higher Priority

Lower Priority

Operator	Example	Result
– (unary negation)	`x = -4 + 3;`	-1
* / %	`x = -4 + 4 % 3 * 13 + 2;`	11
+ –	`x = 6 + 3 – 4 + 6 * 3;`	23

Grouping with Parenthesis

- When parenthesis are used in an expression, the inner most parenthesis are processed first.
- If two sets of parenthesis are at the same level, they are processed left to right.



Combined Assignment Operators

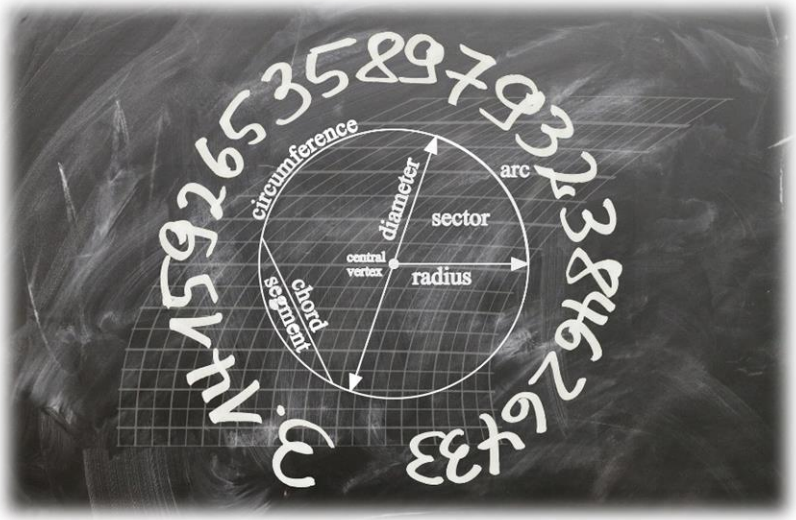
Operator	Example	Equivalent	Value of variable after operation
+=	`x += 5;`	`x = x + 5;`	The old value of **x** plus 5.
-=	`y -= 2;`	`y = y - 2;`	The old value of **y** minus 2
***=**	`z *= 10;`	`z = z * 10;`	The old value of **z** times 10
/=	`a /= b;`	`a = a / b;`	The old value of **a** divided by **b**.
%=	`c %= 3;`	`c = c % 3;`	The remainder of the division of the old value of **c** divided by 3.

Combined Assignment Operators

- Example

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Constants


3.14159265358979323846264338
circumference
diameter
arc
sector
central vertex
radius
chord
segment

Creating Constants

- Constants keep the program organized and easier to maintain.
- Can hold only a single value.
- Constants are declared using the **keyword** **final**.
- Constants need not be initialized when declared; however, they **must be initialized before they are used** or a compiler error will be generated.
- **Once initialized with a value**, constants **cannot be changed programmatically**.
- By convention, constants are **all upper case** and words are separated by the underscore character.

```
final int STUDENT_COUNT = 10;
final double CAL_SALES_TAX = 0.725;
```

The String Class



The String Class

- Java has no primitive data type that holds a series of characters.
- The **String** class from the Java standard library is used for this purpose.
- In order to be useful, a variable must be created to reference a String object.

```
String number;
```

- Notice the **S** in **S**tring is **upper case**.

String Objects

- A variable can be assigned a String literal.

```
String value = "Hello";
```

- Strings are the only objects that can be created in this way.
- A variable can be created using the new keyword.

```
String value = new String("Hello");
```

- This is the method that all other objects must use when they are created.

- Many useful builtin methods:
  contains, startsWith, endsWith, indexOf,substring, split, replace, toUpperCase, toLowerCase, equals

- Use equals (not ==) to compare strings

```
String name = "Ali";
name.equals("Ali");  // --> true
```

String Methods

- **String Length:**

```
String txt = "Be yourself; everyone else is already taken.";
System.out.println("The length of the txt string is: " +
txt.length());
```

- **Uppercase and Lowercase:**

```
String txt = "Hello World";
System.out.println(txt.toUpperCase());   // Outputs "HELLO WORLD"
System.out.println(txt.toLowerCase());   // Outputs "hello world"
```

- **Finding a Character in a String: indexOf() method returns the index (the position) of the first occurrence of a specified text in a string (including whitespace):**

```
String txt = "Please locate where 'locate' occurs!";
System.out.println(txt.indexOf("locate")); // Outputs 7
```

Special Characters

- Because strings must be written within quotes, Java will misunderstand this string, and generate an error:

```
String txt = "We are the so-called "Vikings" from the north.";
```

- The solution to avoid this problem, is to use the backslash escape character.
- The backslash (\) escape character turns special characters into string characters:

Escape character	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

Special Characters

```
String viking = "We are the so-called \"Vikings\" from the north.";
String right = "It\'s alright.";
String lines = "this is one line \nThis is the next line\n";
System.out.println(viking);   //We are the so-called "Vikings" from
the north.
System.out.println(right);    //It's alright.
System.out.println(lines);   //this is one line
                             //This is the next line
```

Using System.out for Screen Output

- Commonly used methods:
  - **print(String *line*)** will send *line* to output
  - **println(String line)** will send line to output, followed by a line break
    println() will send just the line break to output
  - **printf** (formatString, argumentList)

```
double price = 19.8;
String name = "magic apple";
System.out.printf("$%.2f for each %s.", price, name);
```

will output

```
$19.80 for each magic apple
```

  - The formatString contains 2 format specifiers (`%.2f` and `%s`) that match the two arguments (`price` and `name`)
  - The format specifier `"%.2f"` indicates displaying 2 digits after the decimal point

Display formatted Data

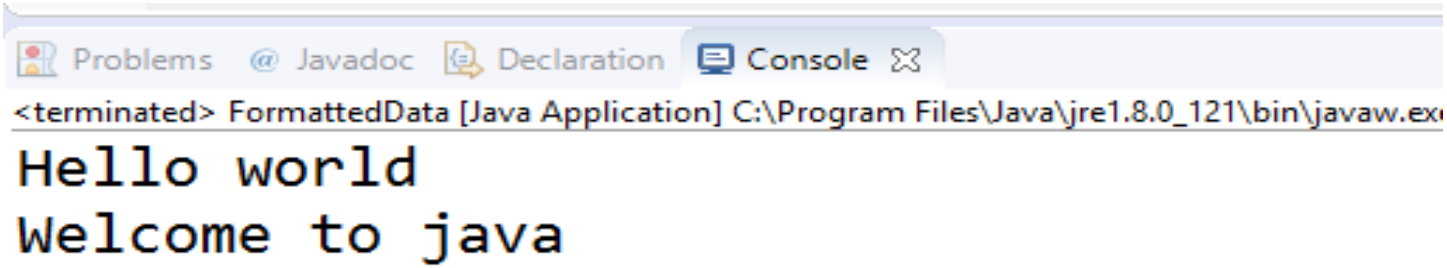
```
public class FormattedData {
 public static void main(String arg[])
 {
  System.out.printf("%s\n%s","Hello world","Welcome to java");
 }

}
```

Format

The format **%s** is a placeholder for a string

\n inserts a line break



Displaying Text with printf

System.out.printf method : f means “formatted”

- Displays formatted data
- Multiple method arguments are placed in a comma-separated list.
- Method **printf’s** first argument is a format string
  - May consist of **fixed text** and format **specifiers**.
  - Fixed text is output as it would be by print or println.
  - Each format specifier is a **placeholder** for a value and specifies the type of data to output.
- **Format specifiers begin with a percent sign (%) and are followed by a character that represents the data type.**
- Format specifier **%s** is a placeholder for a string.

Displaying Text with printf - examples

```
//Number Formatting
int x = 10;
System.out.printf("Formatted output is: %d %d%n", x, -x);
float y = 2.28f;
System.out.printf("Upto 4 decimal places %.4f\n",y);
float z = 3.147293165f;
System.out.printf("Upto 2 decimal places %.2f\n",z);
//Width Specifier, Aligning, Fill With Zeros
System.out.printf("'%5.2f'%n", 2.28);
System.out.printf("'%05.2f'%n", 2.28);
System.out.printf("'%010.2f'%n", 2.28);
//Aligning By default, it is a + which means right
aligned.
System.out.printf("'%10.2f'%n", 2.28);
System.out.printf("'%-10.2f'%n", 2.28);  //Align left
```

```
Formatted output is: 10 -10
Upto 4 decimal places 2.2800
Upto 2 decimal places 3.15
' 2.28'
'02.28'
'0000002.28'
'      2.28'
'2.28      '
```

Common Format Specifiers

Format Specifier	Conversion Applied
%a %A	Floating-point hexadecimal
%b %B	Boolean
%c	Character
%d	Decimal integer
%h %H	Hash code of the argument
%e %E	Scientific notation
%f	Decimal floating-point
%g %G	Uses %e or %f, whichever is shorter
%o	Octal integer
%n	Inserts a newline character
%s %S	String
%t %T	Time and date
%x %X	Integer hexadecimal
%%	Inserts a % sign

Java User Input (Scanner)



The Scanner Class

- To read input from the keyboard we can use the Scanner class.
- The Scanner class is defined in **java.util**, so we will use the following statement at the top of our programs:

```
import java.util.Scanner;
```

- Scanner objects work with System.in
- To create a Scanner object:

```
Scanner keyboard = new Scanner (System.in);
```

```
Scanner myObj = new Scanner(System.in);  // Create a Scanner object
System.out.println("Enter username");
String userName = myObj.nextLine();  // Read user input
System.out.println("Username is: " + userName);  // Output user input
```

The Scanner Class : Input Types

Method	Description
`nextBoolean()`	Reads a `boolean` value from the user
`nextByte()`	Reads a `byte` value from the user
`nextDouble()`	Reads a `double` value from the user
`nextFloat()`	Reads a `float` value from the user
`nextInt()`	Reads a `int` value from the user
`nextLine()`	Reads a `String` value from the user
`nextLong()`	Reads a `long` value from the user
`nextShort()`	Reads a `short` value from the user

The Scanner Class : Input Types Example

```
import java.util.Scanner; //Always needed with input
public class TextScanner {
public static void main(String[] args) {
    Scanner myObj = new Scanner(System.in);
    System.out.println("Enter name, age and salary:");
    // String input
    String name = myObj.nextLine();

    // Numerical input
    int age = myObj.nextInt(); // integer input
    double salary = myObj.nextDouble(); //double input

    // Output input by user
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("Salary: " + salary);
    myObj.close();
  }
}
```

Example-1: Area of a Circle

```
package simple;
import java.util.Scanner;
public class CircleArea {

public static void main(String[] args) {
        final double PI = 3.14159265359;
        double radius, area;
        Scanner input = new Scanner(System.in);
        System.out.println("Enter the radius: ");
        radius = input.nextDouble();
        area = PI * radius * radius;
        System.out.println("Area =" + area);
        }
}
```

```
Enter the radius:
3.7
Area =43.0084034276471
```

Example-2: BMI Calculator

```
package simple;
import java.util.Scanner;
public class BMICalc {

public static void main(String[] args) {
        double weight, height, BMI;
        Scanner input = new Scanner(System.in);
        System.out.print("Enter weight in KGs: ");
        weight = input.nextDouble();
        System.out.print("Enter height in meters: ");
        height = input.nextDouble();
        BMI = weight/(height*height);
        System.out.println("Your BMI = " + BMI);
        }
}
```

```
Enter weight in KGs: 77
Enter height in meters: 1.67
Your BMI = 27.60945175517229
```

Example-3: Adding 3 Integers

```
package simple;
import java.util.Scanner;
public class AddIntegers {

public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int x, y, z, sum;
        System.out.println("Enter 3 integers x, y, z: ");
        x = input.nextInt();
        y = input.nextInt();
        z = input.nextInt();
        sum = x+y+z;
        System.out.println(x + "+" + y + "+" + z + "=" + sum);
        }
}
```

```
Enter 3 integers x, y, z:
4
5
9
4+5+9=18
```

Example-4: Input /Output

```
package simple;
import java.util.Scanner;  // Needed for the Scanner class
public class InputOutput {
public static void main(String[] args)
   {
      String name;   // To hold the user's name
      int age;       // To hold the user's age
      double income; // To hold the user's income
      // Create a Scanner object to read input.
      Scanner input = new Scanner(System.in);
      // Get the user's name.
      System.out.print("What is your name? ");
      name = input.nextLine();
      // Get the user's age.
      System.out.print("What is your age? ");
      age = input.nextInt();
       // Get the user's income
      System.out.print("What is your annual income? ");
      income = input.nextDouble();
      // Display the information back to the user.
      System.out.println("Hello, " + name + ". Your age is " +
                         age + " and your income is QR" + income);
} }
```

```
What is your name? Zeyad F. Ali
What is your age? 50
What is your annual income? 123456.76
Hello, Zeyad F. Ali. Your age is 50 and your income is QR123456.76
```

Example-5: Payroll Calculation

```
package simple;
import java.util.Scanner;  // Needed for the Scanner class
public class PayrollCalc {

public static void main(String[] args) {
            String name;         // To hold a name
      int hours;           // Hours worked
      double payRate;      // Hourly pay rate
      double pay;     // Gross pay
      // Create a Scanner object to read input.
      Scanner input = new Scanner(System.in);
      // Get the user's name.
      System.out.print("What is your name? ");
      name = input.nextLine();
      // Get the number of hours worked this week.
      System.out.print("How many hours did you work this week? ");
      hours = input.nextInt();
      // Get the user's hourly pay rate.
      System.out.print("What is your hourly pay rate? ");
      payRate = input.nextDouble();
      // Calculate the pay.
      pay = hours * payRate;
      // Display the resulting information.
      System.out.println("Hello, " + name);
      System.out.println("Your pay is QR" + pay);
}}
```

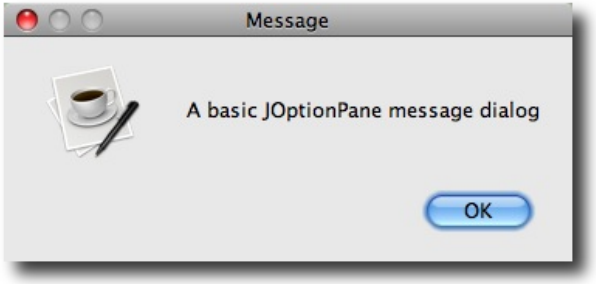
```
What is your name? Zeyad Ali
How many hours did you work this week? 40
What is your hourly pay rate? 50
Hello, Zeyad Ali
Your pay is QR2000.0
```

Dialog Boxes



Dialog Boxes

- A dialog box is a small graphical window that displays a message to the user or requests input.
- A variety of dialog boxes can be displayed using the **JOptionPane** class.
- Two of the dialog boxes are:
  - **Message Dialog** - a dialog box that displays a message.
  - **Input Dialog** - a dialog box that prompts the user for input.





The JOptionPane Class

- The **JOptionPane** class is not automatically available to your Java programs.
- The following statement must be before the program's class header:

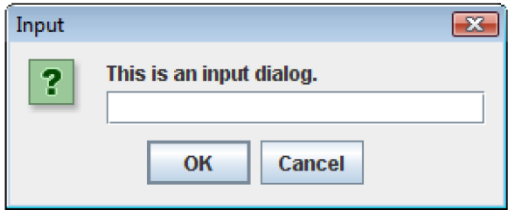
```
import javax.swing.JOptionPane;
```

- This statement tells the compiler where to find the JOptionPane class.

Message dialog



Input dialog



Message Dialogs

- JOptionPane.showMessageDialog method is used to display a message dialog.

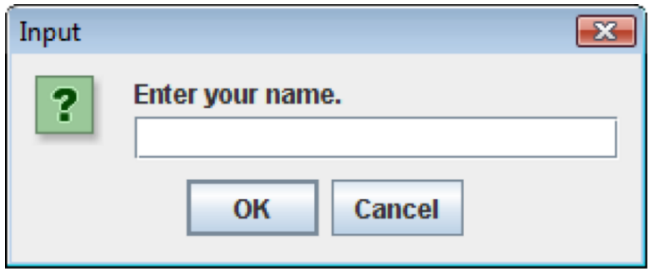
```
JOptionPane.showMessageDialog(null, "Hello World");
```

- The first argument will be discussed later.
- The second argument is the message that is to be displayed.



Input Dialogs

- An input dialog is a quick and simple way to ask the user to enter data.
- The dialog displays a text field, an Ok button and a Cancel button.
- If Ok is pressed, the dialog returns the user's input.
- If Cancel is pressed, the dialog returns null




```
String name;
name = JOptionPane.showInputDialog("Enter your name.");
```

- The argument passed to the method is the message to display.
- If the user clicks on the OK button, name references the string entered by the user. If the user clicks on the Cancel button, name references null.

The System.exit Method

- A program that uses JOptionPane does not automatically stop executing when the end of the main method is reached.
- Java generates a thread, which is a process running in the computer, when a JOptionPane is created.
- If the System.exit method is not called, this thread continues to execute.
- The System.exit method requires an integer argument: **System.exit(0);**
- This argument is an exit code that is passed back to the operating system.
- This code is usually ignored, however, it can be used outside the program to indicate whether the program ended successfully or as the result of a failure.
- The **value 0** traditionally indicates that the program ended successfully.

Converting a String to a Number

- The JOptionPane's showInputDialog method always **returns the user's input as a String**
- A String containing a number, such as "127.89" can be converted to a numeric data type.
- Each of the numeric wrapper classes, (covered in Chapter 10) has a method that converts a string to a number.
- The **Integer** class has a method that converts a string to an **int**,
- The **Double** class has a method that converts a string to a **double**, and etc.
- These methods are known as **parse** methods because their names begin with the word "parse."

The Parse Methods

```
// Store 1 in bVar.
byte bVar = Byte.parseByte("1");
// Store 2599 in iVar.
int iVar = Integer.parseInt("2599");
// Store 10 in sVar.
short sVar = Short.parseShort("10");
// Store 15908 in lVar.
long lVar = Long.parseLong("15908");
// Store 12.3 in fVar.
float fVar = Float.parseFloat("12.3");
// Store 7945.6 in dVar.
double dVar = Double.parseDouble("7945.6");
```

Reading Numeric Values from an Input Dialog

Reading integers:

```
int number;
String str;
str = JOptionPane.showInputDialog("Enter a number.");
number = Integer.parseInt(str);
```

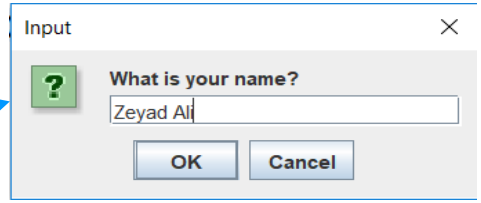
Reading doubles:

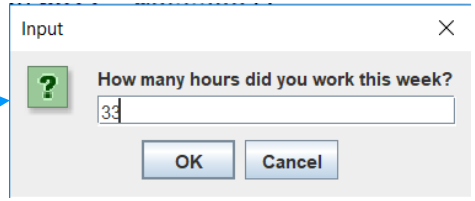
```
double price;
String str;
str = JOptionPane.showInputDialog("Enter the retail price.");
price = Double.parseDouble(str);
```

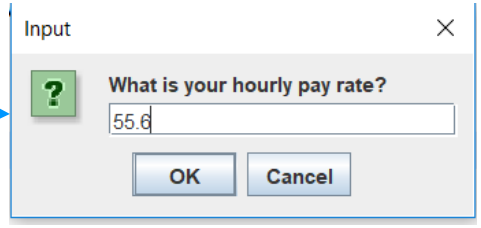
Example-6: Payroll Calculation Using Dialogs

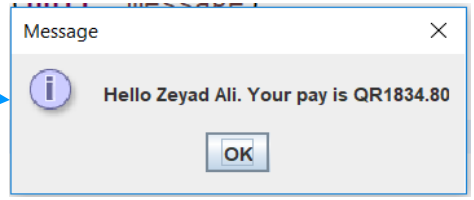
```
package simple;
import javax.swing.JOptionPane;
public class InputOutputDialog {

public static void main(String[] args) {
String inputString; // For reading input
      String name;        // The user's name
      int hours;          // The number of hours worked
      double payRate;     // The user's hourly pay rate
      double pay;    // The user's gross pay
      // Get the user's name.
      name = JOptionPane.showInputDialog("What is your name? ");
      // Get the hours worked.
      inputString =      JOptionPane.showInputDialog("How many hours did you work this week? ");
      // Convert the input to an int.
      hours = Integer.parseInt(inputString);
      // Get the hourly pay rate.
      inputString = JOptionPane.showInputDialog("What is your hourly pay rate? ");
     // Convert the input to a double.
      payRate = Double.parseDouble(inputString);
      // Calculate the pay.
      pay = hours * payRate;
       // Display the results.
               String message = String.format("Hello %s. Your pay is QR%.2f", name, pay);
       JOptionPane.showMessageDialog(null, message);
     // End the program.
      System.exit(0);
}}
```









Java Type Casting

- Type casting is when you assign a value of one primitive data type to another type.
- In Java, there are two types of casting:
  - **Widening/Promotion Casting (automatically)** - converting a smaller type to a larger type size

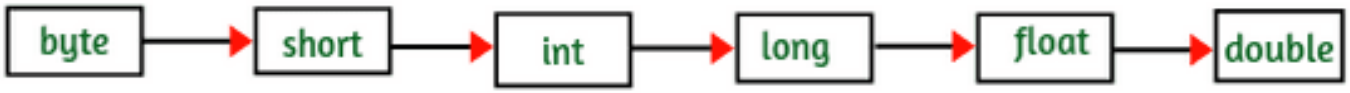
    byte -> short -> char -> int -> long -> float -> double
  - **Narrowing/Demotion Casting (manually)** - converting a larger type to a smaller size type

    double -> float -> long -> int -> char -> short -> byte

Widening Casting

- **Widening casting is done automatically** when passing a smaller size type to a larger size type:

```
int k = 3;
double c = k; // Automatic casting: int to double
System.out.println(k);      // Outputs 3
System.out.println(c);   // Outputs 3.0
```



Narrowing Casting

- Narrowing casting must be done manually by placing the type in parentheses in front of the value:

```
double myDouble = 5.67;
int myInt = (int) myDouble; // Manual casting: double to int
System.out.println(myDouble);   // Outputs 5.67
System.out.println(myInt);      // Outputs 5
```

Packages and the import statement

- **Java Package:**
  - A method to categorize classes and interfaces
  - Package name comes first in your java file
- **Import statements:**
  - Import statement is a way of giving the proper location for the compiler to find that particular class.
  - In java if a fully qualified name, which includes the package and the class name, is given then the compiler can easily locate the classes.

Built-in Packages



- Java fundamental classes are in *java.lang*, classes for reading and writing (input and output) are in *java.io*, lists and collections in *java.util* and so on.
- To use a class from a package, first **import** it. E.g.,

```
import java.util.Scanner;
```

Resources

- More info and exercises @
  - https://www.w3schools.com/java/
  - https://www.javatpoint.com/java-tutorial
  - https://www.w3resource.com/java-exercises/basic/index.php